- Вступление
- Платформа и запуск
  - JDK, JRE, JVM
  - Метод main и запуск
  - Пакеты и import
- Типы данных и операторы
  - Примитивные и ссылочные типы
  - Значения по умолчанию
  - Литералы
  - Приведение типов
  - Обёртки и автоупаковка
  - Передача аргументов — всегда по значению
  - Stack и Heap — где лежат данные
  - Операторы
- String
  - Пул строк и intern()
  - Почему String неизменяемый
  - StringBuilder и StringBuffer
  - Строки в switch и полезные методы
  - Пароли в массиве char
- Класс изнутри
  - Модификаторы доступа
  - final
  - static
  - Конструкторы

    - Блоки инициализации и порядок создания объекта
    - Неизменяемый класс (immutable)
- Наследование и абстракция
    - abstract
    - Интерфейс
    - Абстрактный класс vs интерфейс
    - Переопределение и перегрузка
- Вложенные классы
- Перечисления — enum
- Object — equals, hashCode, toString, clone
    - equals и ==
    - hashCode и связь с equals
    - toString
    - Клонирование
    - finalize — устарел
- Исключения
    - Иерархия
    - Checked и unchecked
    - throw, throws, своё исключение
    - try / catch / finally
    - try-with-resources
    - Исключение из main
- Generics
    - Обобщённые классы и методы
    - Стирание типов (type erasure)
    - Инвариантность и wildcards

- Java 8 — лямбды, функциональные интерфейсы, Optional, Date-Time API
  - Лямбда-выражения
  - Ссылки на методы
  - Функциональные интерфейсы
  - Optional
  - Новые методы Map
  - Date-Time API (java.time)
  - Прочее из Java 8
- Java 9–25 — var, record, sealed, pattern matching, text blocks
  - var
  - record
  - sealed
  - switch-выражения и pattern matching
  - Text blocks
  - Модули (JPMS)
- Reflection и аннотации
  - Reflection
  - Аннотации
- Прочее — varargs, assert
  - Varargs
  - assert
- Числа и точность — BigDecimal
  - Почему double нельзя для денег
  - BigDecimal
- Вопросы к собеседованию

Вступление

Конспект по Java Core: язык, типы, классы и ООП, `Object`, исключения, generics и то, что добавляли в Java 8 и новее. Ориентир — **Java 21 / 25 (LTS)**.

Темы идут от общего к частному:

- **Платформа, типы, String** — из чего состоит программа и как хранятся данные;
- **Класс, наследование, вложенные классы, enum** — как устроены классы и ООП в Java;
- **Object, исключения, generics** — общие механизмы, на которых стоит весь JDK;
- **Java 8** и **Java 9–25** — что появилось в языке по версиям;
- **Reflection, аннотации, BigDecimal** — то, что нужно для Spring и финтеха.

Что вынесено в отдельные конспекты: устройство памяти, GC и ClassLoader — **JVM**; коллекции и устройство `HashMap` — **Collections**; Stream API — **Java Stream**; ввод-вывод — **Java IO**; сериализация — **serialization**; многопоточность — **Java Concurrency**.

Платформа и запуск

JDK, JRE, JVM

- **JVM (Java Virtual Machine)** — виртуальная машина, которая исполняет байт-код (`.class`): загружает классы,

интерпретирует и JIT-компилирует код, управляет памятью и GC. Своя реализация под каждую ОС — поэтому байт-код переносим.

- **JRE (Java Runtime Environment)** — JVM + стандартная библиотека классов. Достаточно, чтобы **запускать** программы.
- **JDK (Java Development Kit)** — JRE + инструменты разработчика: `javac`, `jar`, `javadoc`, `jshell`, `jlink`, `jcmd` и т.д. Нужен, чтобы **писать и собирать** программы.

```
JDK ⊃ JRE ⊃ JVM
.java --javac--> .class (байт-код) --JVM--> машинный код
```

С Java 11 отдельный JRE больше не поставляется: ставят JDK, а для продакшена собирают урезанный рантайм через `jlink` или берут готовый Docker-образ.

Подробно про устройство JVM — в конспекте **JVM**.

Метод main и запуск

Метод `main` — точка входа в программу. Классов с `main` в приложении может быть несколько — запускается тот, который указан при старте. Если `main` нет, код скомпилируется, но при запуске будет `Error: Main method not found`.

```
public class App {
    public static void main(String[] args) { }
}
```

Современные варианты:

```
// Java 25 (JEP 512): компактный файл и instance main — без класса, public, static и args
void main() {
    IO.println("Hello");
}
```

```
javac App.java && java App   # классика: компиляция + запуск
java App.java                 # Java 11+: запуск исходника без явной компиляции
```

Пакеты и import

Пакет (package) — пространство имён для классов: группирует связанные классы и задаёт границу доступа `default` (package-private). Имя пакета соответствует структуре папок: `com.bank.payments` → `com/bank/payments/`.

```
package com.bank.payments;

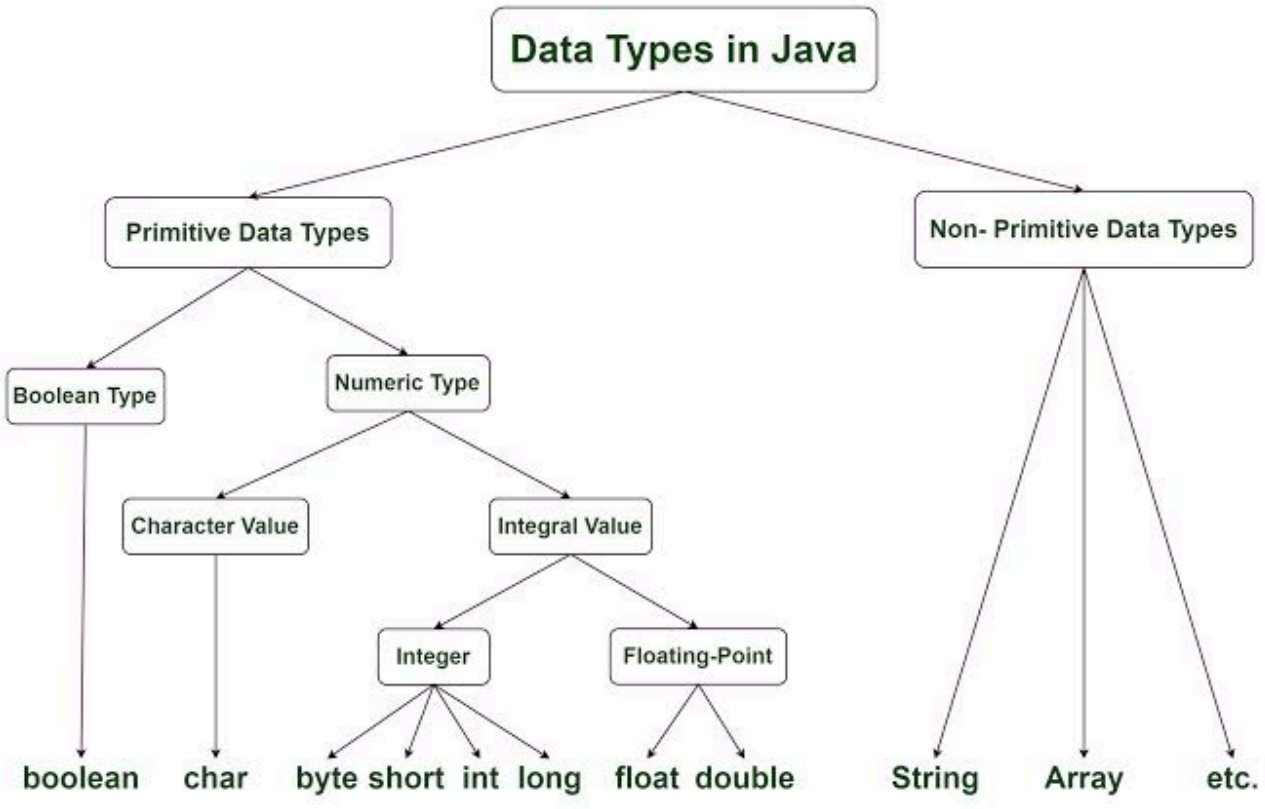
import java.util.List;                    // один класс
import java.util.*;                        // все классы пакета (не подпакеты)
import static java.lang.Math.max;          // статический член — вызываем max(a, b) без Math.
```

- `java.lang` импортируется автоматически (`String`, `Object`, `Math`, `Integer` ...).

- `import` ничего не загружает и не влияет на производительность — это только сокращение имён для компилятора.

Типы данных и операторы

Примитивные и ссылочные типы



Примитивный тип — один из 8 встроенных типов, переменная которого хранит само значение. Пишутся с маленькой буквы.

Ссылочный тип — класс, интерфейс, массив, enum, record. Переменная хранит **ссылку** на объект в куче (или `null`). Пишутся с большой буквы.

Тип	Размер	Диапазон / примечание	Обёртка
`byte`	1 байт	−128 … 127	`Byte`
`short`	2 байта	−32 768 … 32 767	`Short`
`int`	4 байта	≈ ±2,1 млрд	`Integer`
`long`	8 байт	≈ $\pm 9{,}2 \cdot 10^{18}$	`Long`
`float`	4 байта	~7 значащих цифр	`Float`
`double`	8 байт	~15–16 значащих цифр	`Double`
`char`	2 байта	0 … 65 535, символ UTF-16 (беззнаковый)	`Character`
`boolean`	не определён спецификацией	`true` / `false`	`Boolean`

Зачем нужны обёртки (`Integer` , `Double` ...):

1. **Коллекции и generics** — `List<Integer>` можно, `List<int>` нельзя.
2. `null` — можно выразить «значения нет» (например, пустая колонка в БД).
3. **Утилиты** — `Integer.parseInt("42")` , `Integer.MAX_VALUE` , `Character.isDigit(c)` .

Значения по умолчанию

Значения по умолчанию получают **только поля класса и элементы массива**. Локальную переменную нужно инициализировать явно — иначе ошибка компиляции.

Тип	По умолчанию
`byte`, `short`, `int`	`0`
`long`	`0L`
`float` / `double`	`0.0f` / `0.0d`
`char`	`'\u0000'`
`boolean`	`false`
любой ссылочный (в т.ч. `String`)	`null`

```
class Account {
    int balance;          // 0
    String owner;         // null
    void m() {
        int x;
        // System.out.println(x);   // ошибка
компиляции: variable x might not have been initialized
    }
}
```

Литералы

Литерал — значение, записанное прямо в коде: `42`, `3.14`, `'a'`, `"Hello"`, `true`, `null`. Тип литерала определяется формой записи.

```
int dec = 42;
int bin = 0b1010_1010;      // двоичный (Java 7+)
int hex = 0xFF;             // шестнадцатеричный
int oct = 010;              // восьмеричный! = 8, а не
10 — частая ловушка
long big = 1_000_000_000L;  // _ для читаемости, L —
```

```
long
float f = 3.14f;              // без f литерал 3.14 — это
double
char c = 'A';              // 'A'
String text = """
        многострочная
        строка""";          // text block (Java 15)
```

Приведение типов

Приведение типов (casting) — преобразование значения одного типа в другой. Java статически типизирована, поэтому одни приведения компилятор делает сам, а другие нужно писать явно.

Вид	Направление	Как	Риск
Расширение примитива (widening)	`byte → short → int → long → float → double`, `char → int`	автоматически	нет (кроме точнос `long → float/double`)
Сужение примитива (narrowing)	`double → int`, `int → byte` …	явно: `(byte) x`	старшие биты отбрасываются
Восходящее ссылочное (upcasting)	потомок → предок	автоматически	нет
Нисходящее ссылочное (downcasting)	предок → потомок	явно: `(Child) p`	`ClassCastExcept`

Вид	Направление	Как	Риск
Запрещённое	примитив ↔ ссылка (кроме boxing), несвязанные классы	—	ошибка компиляц

```
int i = 130;
byte b = (byte) i;            // -126: остались только
младшие 8 бит
int t = (int) 3.99;           // 3: дробная часть
отбрасывается, не округляется

Object o = "Hello";           // upcasting
String s = (String) o;        // downcasting — ок
Integer n = (Integer) o;      // компилируется, но
ClassCastException в рантайме
```

При приведении ссылок сам объект не меняется — меняется только тип ссылки, через которую к нему обращаются.

Безопасный downcasting — через `instanceof` с pattern matching (Java 16):

```
if (o instanceof String str) {   // проверка +
приведение + новая переменная
    System.out.println(str.length());
}
```

Ловушки арифметики:

```
byte a = 10;
// a = a + 5;          // ошибка: byte + int = int
a += 5;                // ок: составное присваивание
делает неявный (byte)

int x = Integer.MAX_VALUE + 1;   // переполнение молча:
-2147483648
Math.addExact(Integer.MAX_VALUE, 1);   // бросит
ArithmeticException

int e = 5 / 0;         // ArithmeticException
double inf = 5.0 / 0; // Infinity
double nan = 0.0 / 0; // NaN
```

Обёртки и автоупаковка

Автоупаковка (autoboxing) — автоматическое превращение примитива в объект-обёртку: `Integer x = 5;` компилятор превращает в `Integer.valueOf(5)`. **Распаковка (unboxing)** — обратное: `int y = x;` → `x.intValue()`.

Происходит при присваивании, передаче аргумента, добавлении в коллекцию, арифметике с обёртками.

```
List<Integer> list = new ArrayList<>();
list.add(5);              // boxing
int first = list.get(0);  // unboxing
```

Кэш обёрток. `Integer.valueOf` возвращает закэшированные объекты для значений **−128…127** (так же у `Byte`, `Short`, `Long`, `Character` 0…127, `Boolean`). Поэтому `==` на обёртках работает «иногда»:

```
Integer a = 100, b = 100;
a == b;          // true  — один объект из кэша
Integer c = 200, d = 200;
c == d;          // false — два разных объекта
c.equals(d);     // true  — обёртки всегда сравниваем
через equals
```

Ловушки:

```
Integer n = null;
int v = n;                    // NullPointerException
при распаковке

Long l = 5L;
l.equals(5);                  // false: 5 упакуется в
Integer, а не в Long

boolean flag = true;
Integer i = null;
int r = flag ? i : 0;         // NPE: тернарник
распакует i в int
```

Конструкторы обёрток (`new Integer(5)`) устарели и помечены к удалению (Java 9 / 16) — используем `valueOf` или автоупаковку.

Передача аргументов — всегда по значению

В Java аргументы **всегда передаются по значению**: метод получает копию. Для примитива копируется значение, для объекта — **копия ссылки**.

Следствие: через копию ссылки можно изменить **содержимое** объекта, но нельзя заменить сам объект у вызывающего кода.

```
void change(StringBuilder sb, int n) {
    sb.append("!");          // виден снаружи — меняем объект
    sb = new StringBuilder(); // не виден — меняем только локальную копию ссылки
    n = 100;                  // не виден — копия примитива
}
```

Stack и Heap — где лежат данные

- **Stack (стек потока)** — у каждого потока свой. На каждый вызов метода создаётся фрейм с локальными переменными (примитивы и ссылки) и удаляется при выходе. Работает по LIFO, быстрый, маленький. Переполнение — `StackOverflowError` (обычно бесконечная рекурсия). Размер — `-Xss`.
- **Heap (куча)** — общая для всех потоков. Здесь живут **все объекты** (всё, что создано через `new`, массивы, строки). Память освобождает GC. Нехватка — `OutOfMemoryError: Java heap space`. Размер — `-Xms` / `-Xmx`.

«Примитивы в стеке, объекты в куче» — **не совсем так**: примитивное **поле** объекта лежит в куче вместе с объектом. В стеке только локальные переменные и параметры методов.

```
class Account {
    long balance;                    // в куче,
```

```
внутри объекта Account
}
void m() {
    int count = 0;                          // в стеке
    Account acc = new Account();            // ссылка acc в
стеке, объект — в куче
}
```

Нюанс: JIT через escape analysis может вообще не создавать объект, который не «убегает» из метода, и разложить его поля по регистрам (scalar replacement).

Подробно про области памяти и GC — в конспекте **JVM**.

Операторы

Группа	Операторы	Важное
Арифметические	`+ - * / % ++ --`	`int / int` = целое; `%` сохраняет знак делимого: `-7 % 3 == -1`
Сравнение	`== != < > <= >=`	для ссылок `==` сравнивает **адреса**, не содержимое
Логические	`&& || !`	**сокращённые**: правая часть не вычисляется, если результат уже ясен
Логические полные	`& | ^` на `boolean`	вычисляют **обе** части; `^` — XOR
Побитовые	`& | ^ ~`	работают с битами целых чисел
Сдвиги	`<< >> >>>`	`<<` ×2ⁿ, `>>` ÷2ⁿ со знаком, `>>>` — без знака

Группа	Операторы	Важное
		(заполняет нулями)
Присваивание	`= += -= *= &= |= ^= <<= >>= ...`	составные делают неявное приведение
Тернарный	`cond ? a : b`	выражение, а не оператор: возвращает значение
Проверка типа	`instanceof`	`null instanceof X` → `false`

```
if (user != null && user.isActive()) { }   // && защищает от NPE
int flags = READ | WRITE;                  // битовые флаги
boolean canWrite = (flags & WRITE) != 0;
-8 >> 1;    // -4
-8 >>> 1;   // 2147483644 — знак не сохраняется

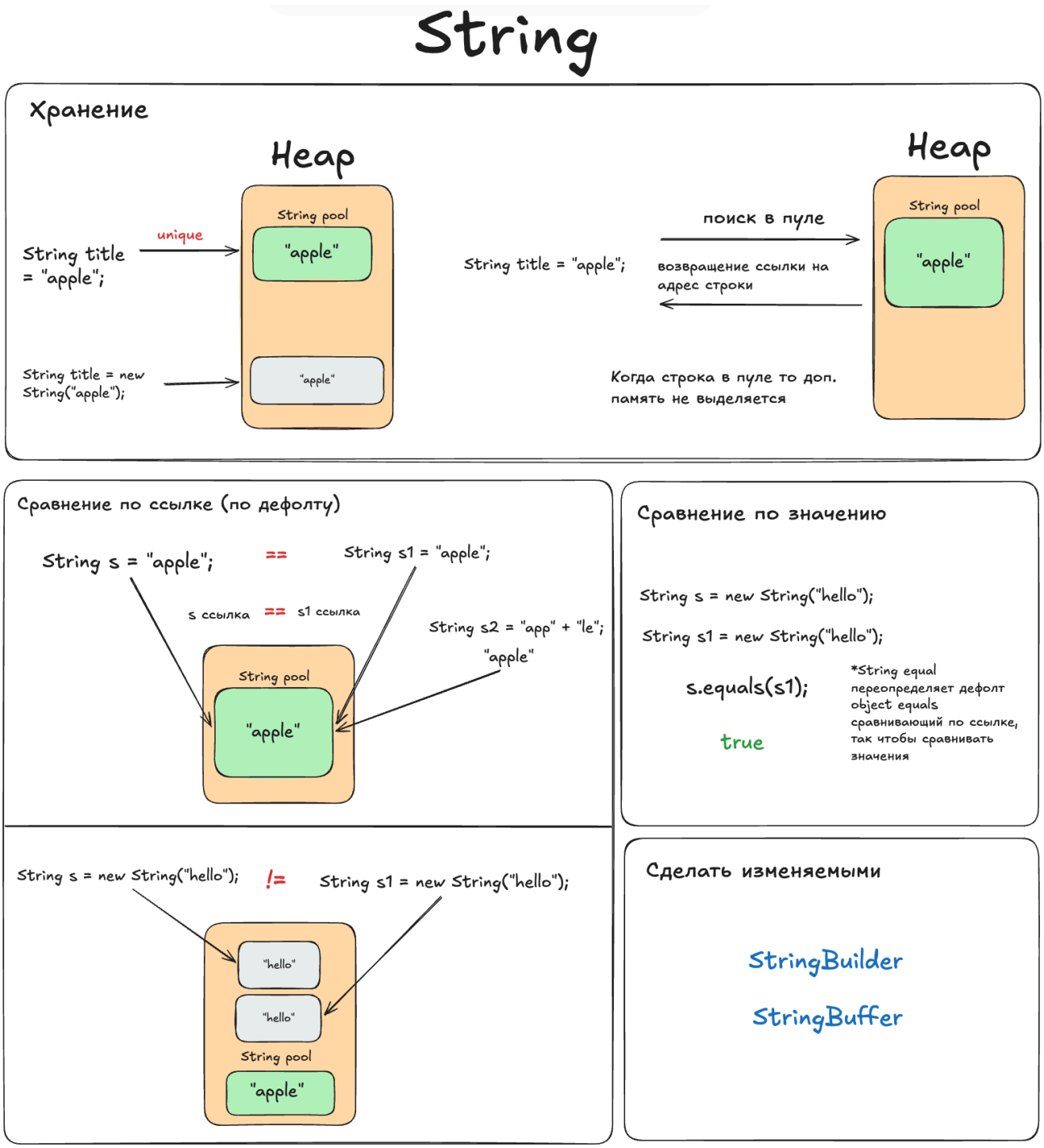
String label = balance >= 0 ? "ok" : "debt";
Object r = true ? 1 : 2.0;   // 1.0 — оба операнда приводятся к общему типу double
```

Сдвиг маскирует количество бит: для `int` берутся младшие 5 бит, поэтому `1 << 32 == 1`.

String

String — неизменяемая (immutable) последовательность символов; класс `final`. Любая «модификация» (`concat`,

`replace`, `toUpperCase`, `+`) создаёт **новую** строку.



Пул строк и intern()

Пул строк (String pool) — область в куче (с Java 7), где JVM хранит по одному экземпляру каждого строкового литерала. Одинаковые литералы указывают на один объект.

```
String a = "apple";
String b = "apple";
String c = new String("apple");   // всегда новый объект в куче, мимо пула
String d = "app" + "le";          // константа времени компиляции → тот же литерал из пула

a == b;            // true  — один объект в пуле
a == c;            // false — разные объекты
a == d;            // true
a == c.intern();   // true  — intern() возвращает экземпляр из пула (добавляет, если нет)
a.equals(c);       // true  — строки всегда сравниваем через equals
```

Конкатенация с переменной (`s1 + s2`) вычисляется в рантайме и даёт новый объект, не из пула.

Почему String неизменяемый

- **Пул строк** возможен только потому, что строку нельзя поменять через одну из ссылок.
- **Безопасность** — строки передают URL, логины, имена классов для загрузки; после проверки их нельзя подменить.
- **Потокобезопасность** без синхронизации.
- **Кэш** **`hashCode()`** — считается один раз и хранится в объекте, поэтому `String` — идеальный ключ `HashMap` : хэш не меняется и не пересчитывается.

`final` у класса нужен, чтобы наследник не сломал неизменяемость.

StringBuilder и StringBuffer

	`String`	`StringBuilder`	`StringBuffe`
Изменяемость	нет	да	да
Потокобезопасность	да (immutable)	нет	да (`synchroniz`
Когда	почти всегда	сборка строки в цикле	легаси; на практике не нужен

```
// плохо: в цикле на каждой итерации создаётся новая строка — O(n²)
String s = "";
for (String part : parts) s += part;

// хорошо
var sb = new StringBuilder();
for (String part : parts) sb.append(part).append(',');

// ещё лучше для простых случаев
String joined = String.join(",", parts);
```

Одиночный `a + b + c` компилятор оптимизирует сам (с Java 9 — через `invokedynamic` / `StringConcatFactory`), `StringBuilder` нужен именно в циклах.

Строки в switch и полезные методы

С Java 7 строки работают в `switch` — сравнение через `equals()` (с учётом регистра), `null` даёт NPE.

```
String type = switch (command) {          // switch-выражение, Java 14
    case "deposit", "topup" -> "IN";
    case "withdraw" -> "OUT";
    default -> throw new IllegalArgumentException(command);
};
```

```
" a ".strip();          // "a" — Unicode-aware (Java 11), лучше trim()
"".isBlank();           // true — пустая или только пробелы (Java 11)
"ab".repeat(3);         // "ababab" (Java 11)
"a\nb".lines();         // Stream<String> (Java 11)
"%s: %d".formatted("x", 1);   // (Java 15)
```

Пароли в массиве char

Строка с паролем живёт в памяти, пока её не соберёт GC, и её нельзя затереть. Массив `char[]` можно очистить сразу после использования — поэтому API вроде `Console.readPassword()` и `KeyStore` работают с `char[]`.

```
char[] password = console.readPassword();
try {
    authenticate(password);
} finally {
    Arrays.fill(password, '\0');   // затираем сразу
}
```

Класс изнутри

Класс — шаблон объектов: поля (состояние), методы (поведение), конструкторы (создание). **Объект** — экземпляр класса в куче.

Модификаторы доступа

Модификатор доступа задаёт, из какого кода виден класс или его член.

Модификатор	Класс	Пакет	Наследник (в др. пакете)	Все
`private`	✓			
(default) package-private	✓	✓		
`protected`	✓	✓	✓	
`public`	✓	✓	✓	✓

- Класс верхнего уровня может быть только `public` или package-private.
- `private` работает на уровне **класса**, а не объекта: метод класса видит `private`-поля другого экземпляра того же класса (так работают `equals` и конструктор копирования).
- Снаружи до `private` можно добраться только через рефлексию (`setAccessible(true)`), и то — если модуль это разрешает.

final

`final` — «нельзя изменить» в зависимости от того, к чему применён:

К чему	Значит
переменная / параметр	присвоить можно один раз
поле	присвоить один раз — при объявлении, в блоке инициализации или в конструкторе
метод	нельзя переопределить
класс	нельзя наследовать (`String`, `Integer`)

```
final List<String> list = new ArrayList<>();
list.add("ok");           // можно: final запрещает
менять ссылку, а не объект
// list = new ArrayList<>();   // нельзя
```

Effectively final — переменная без `final`, которой ни разу не переприсваивали значение. Только такие локальные переменные можно использовать в лямбдах и анонимных классах.

static

`static` — член принадлежит **классу**, а не объекту: существует в одном экземпляре, доступен без создания объекта.

К чему	Значит
поле	одно на класс, общее для всех экземпляров
метод	вызывается через класс; нет `this`, не видит нестатические поля напрямую

К чему	Значит
блок `static {}`	выполняется один раз при инициализации класса
вложенный класс	не привязан к экземпляру внешнего (см. Вложенные классы)
`import static`	импорт статических членов

- Статический метод **нельзя переопределить** — можно только **скрыть** (hiding): метод с той же сигнатурой в наследнике. Вызов выбирается по типу **ссылки** на этапе компиляции, полиморфизма нет.
- Перегрузить статический метод можно (и статическим, и нестатическим — это просто разные сигнатуры).
- `abstract static` — нельзя: абстрактный метод должен быть переопределён, а статический переопределить нельзя.

```
class Parent { static String name() { return "Parent";
} }
class Child extends Parent { static String name() {
return "Child"; } }

Parent p = new Child();
p.name();   // "Parent" — выбран по типу ссылки, а не
объекта
```

Конструкторы

Конструктор — специальный метод инициализации объекта: имя совпадает с именем класса, нет возвращаемого типа,

вызывается через `new`.

- **По умолчанию** — компилятор создаёт `public ClassName() {}` (с доступом как у класса), **только если** в классе нет ни одного конструктора.
- **С параметрами** — принимает значения для полей.
- **Копирования** — принимает объект того же класса и копирует его состояние (лучшая альтернатива `clone()`).
- **Приватный** — запрещает `new` снаружи: утилитные классы, синглтоны, фабричные методы.

```
public class Money {
    private final long amount;
    private final String currency;

    public Money(long amount, String currency) {
        this.amount = amount;
        this.currency = currency;
    }
    public Money(long amount) { this(amount, "RUB"); }
// this(...) — другой конструктор
    public Money(Money other) { this(other.amount,
other.currency); }   // копирования
}

public final class MathUtils {
    private MathUtils() { }   // утилитный класс —
экземпляры не нужны
}
```

Первой строкой конструктора всегда идёт `this(...)` или `super(...)`; если не написано — компилятор вставит `super()`.

Поэтому если у родителя нет конструктора без аргументов, наследник обязан явно вызвать `super(args)`.

Блоки инициализации и порядок создания объекта

Блок инициализации — код в `{ }` внутри класса, выполняется при каждом создании объекта перед телом конструктора. **Статический блок** `static { }` — выполняется один раз при инициализации класса (первое обращение к классу: `new`, вызов статического метода, обращение к статическому полю не-константе).

Зачем: сложная инициализация статических констант (кэши, загрузка драйверов), общий код для нескольких конструкторов.

Порядок при `new Child()`:

```
1. static-поля и static-блоки Parent   (по порядку в
коде, один раз за жизнь класса)
2. static-поля и static-блоки Child    (один раз)
3. поля и {} блоки Parent              (по порядку в
коде)
4. конструктор Parent
5. поля и {} блоки Child
6. конструктор Child
```

```
class Parent {
    static { System.out.println("1 static Parent"); }
    { System.out.println("3 init Parent"); }
    Parent() { System.out.println("4 ctor Parent"); }
}
```

```
class Child extends Parent {
    static { System.out.println("2 static Child"); }
    { System.out.println("5 init Child"); }
    Child() { System.out.println("6 ctor Child"); }
}
// new Child(); new Child(); → 1 2 3 4 5 6 3 4 5 6
```

Ловушка: если конструктор родителя вызывает переопределённый метод, тот выполнится в наследнике **до** инициализации его полей — увидит `null` / `0`.

Исключение в блоке инициализации:

- в **статическом** блоке → `ExceptionInInitializerError`; класс помечается как сломанный, и все следующие обращения к нему дают `NoClassDefFoundError`;
- в **обычном** блоке → исключение пробрасывается из `new`, как из конструктора. Checked-исключение в нём можно бросить, только если его объявляют в `throws` **все** конструкторы.

Неизменяемый класс (immutable)

Неизменяемый объект — состояние не меняется после создания. Потокобезопасен без синхронизации, безопасен как ключ `HashMap`, его можно свободно кэшировать и передавать.

Рецепт:

1. класс `final` (или приватный конструктор + фабрика);
2. все поля `private final`;
3. нет сеттеров;
4. изменяемые поля — **защитное копирование** на входе и выходе.

```
public final class Transfer {
    private final String id;
    private final List<String> tags;

    public Transfer(String id, List<String> tags) {
        this.id = id;
        this.tags = List.copyOf(tags);    // копия +
неизменяемая
    }
    public List<String> tags() { return tags; }   //
уже неизменяемая — можно отдать как есть
}
```

С Java 16 для таких классов есть `record` — см. Java 9–25 — var, record, sealed, pattern matching, text blocks.

Наследование и абстракция

Наследование — класс-потомок (`extends`) получает поля и методы предка и может их расширять и переопределять. В Java у класса **один** родитель, но **сколько угодно** интерфейсов (`implements`). Все классы неявно наследуют `Object` .

Полиморфизм — вызов метода через ссылку на предка выполняет реализацию **фактического** класса объекта (позднее, динамическое связывание). Работает для нестатических, неприватных, нефинальных методов.

```
Account acc = new SavingsAccount();
acc.monthlyFee();   // вызовется
```

```
SavingsAccount.monthlyFee()
```

abstract

Абстрактный класс (`abstract class`) — нельзя создать через `new`; служит базой для наследников. Может содержать всё, что обычный класс: поля, конструкторы, реализованные методы.

Абстрактный метод — объявлен без тела. Класс с хотя бы одним абстрактным методом обязан быть абстрактным; первый конкретный наследник обязан реализовать все абстрактные методы.

```
abstract class Payment {
    protected final long amount;
    protected Payment(long amount) { this.amount =
amount; }   // конструктор есть, хоть new нельзя

    abstract long fee();
// реализуют наследники
    long total() { return amount + fee(); }
// шаблонный метод
}
```

Несовместимо с `abstract`: `final` (нельзя переопределить), `static`, `private` (не видно наследнику).

Интерфейс

Интерфейс — контракт: набор методов, которые обязан реализовать класс. Описывает **что умеет** объект, а не как он устроен.

Модификаторы по умолчанию:

- поля — всегда `public static final` (константы);
- абстрактные методы — `public abstract`;
- `default`-методы (Java 8) — метод с реализацией, наследуется;
- `static`-методы (Java 8) — вызываются только через интерфейс: `Comparator.naturalOrder()`;
- `private`-методы (Java 9) — общий код для `default`-методов.

```
interface Fee {
    long BASE = 10;
// public static final
    long calc(long amount);
// public abstract
    default long withBase(long amount) { return BASE +
calc(amount); }
    static Fee free() { return a -> 0; }
}
```

- Метод интерфейса не может быть `final`: абстрактный обязан быть переопределён, `default` задуман именно для переопределения.
- Если класс получает два одинаковых `default`-метода из разных интерфейсов — он обязан переопределить метод и может явно выбрать реализацию: `A.super.method()`.
- Метод класса всегда побеждает `default` из интерфейса.

Маркерный интерфейс — интерфейс без методов, метка для JVM или библиотеки: `Serializable`, `Cloneable`, `RandomAccess`.

Сегодня для меток чаще используют аннотации.

Абстрактный класс vs интерфейс

	Абстрактный класс	**Интерфейс**
Наследование	только один	сколько угодно
Состояние (поля экземпляра)	да	нет, только константы
Конструкторы	да	нет
Модификаторы методов	любые	`public` (и `private` с Java 9)
Смысл	«является» — общая база с состоянием	«умеет» — роль, способность

Выбор:

- **интерфейс** — по умолчанию: описать роль (`Comparable` , `Repository`), дать нескольким несвязанным классам общий контракт;
- **абстрактный класс** — когда нужны общее состояние, конструктор, защищённые методы или шаблонный метод для близких классов.

Переопределение и перегрузка

Переопределение (overriding) — наследник даёт свою реализацию метода предка с **той же сигнатурой**. Выбор метода — в рантайме по типу объекта.

Перегрузка (overloading) — несколько методов с одним именем и **разными параметрами** в одном классе. Выбор — при компиляции по типам аргументов.

Что меняем при переопределении	**Можно?**
Имя метода, типы / количество / порядок параметров	нет — это уже перегрузка
Имена параметров	да, ни на что не влияет
Модификатор доступа	только **расширить** (`protected → public`), сузить нельзя
Возвращаемый тип	тот же или **подтип** (ковариантный возврат); для примитивов — только тот же
`throws` checked-исключений	убрать, сузить до подклассов; добавить новые или более широкие — нельзя
`throws` unchecked	любые
`static` / `final` / `private` метод предка	переопределить нельзя

Логика правил — **принцип подстановки Лисков**: наследник должен работать везде, где ожидается предок, поэтому не может требовать больше (новые исключения) или давать меньше (закрытый доступ).

```
class Repo {
    protected Object find(long id) throws IOException {
... }
```

```
}
class UserRepo extends Repo {
    @Override
    public User find(long id) throws
FileNotFoundException { ... }   // шире доступ, уже тип
и исключение
}
```

- `@Override` — всегда ставим: компилятор проверит, что метод действительно переопределяет.
- `super.method()` — вызов реализации предка; `super(...)` — вызов конструктора предка.

Вложенные классы

Вложенный класс (nested class) — класс, объявленный внутри другого класса. Нужен, когда класс используется только своим внешним классом: группирует код, усиливает инкапсуляцию (вложенный видит `private` внешнего и наоборот).

Вид	Где объявлен	Связь с экземпляром внешнего	Типичное применение
Статический вложенный (`static class`)	в теле класса	нет	Builder, `Map.Entry`, вспомогательны DTO
Внутренний (inner, без `static`)	в теле класса	хранит неявную	итератор коллекции, доступ к полям

Вид	Где объявлен	Связь с экземпляром внешнего	Типичное применение
		ссылку `Outer.this`	внешнего объекта
Локальный	внутри метода / блока	есть, если метод нестатический	вспомогательны класс для одног метода
Анонимный	в выражении `new Интерфейс() { ... }`	есть, если создан в нестатическом контексте	одноразовая реализация; сейчас обычно заменяется лямбдой

```
public class Bank {
    private final String name = "B";

    static class Builder { }                    // new
Bank.Builder()

    class Branch {                              //
bank.new Branch()
        String info() { return Bank.this.name; }   //
доступ к полю внешнего объекта
    }

    void process() {
        int limit = 100;                        //
effectively final
        class Validator {                       //
локальный
            boolean ok(int x) { return x < limit; }
```

```
        }
        Runnable r = new Runnable() {          //
анонимный
            public void run() {
System.out.println(limit); }
        };
    }
}
```

Особенности:

- **Статический вложенный** — по сути обычный класс верхнего уровня в пространстве имён внешнего; видит только статические члены внешнего. «Статический класс» в Java — это именно он: верхнеуровневый класс `static` быть не может.
- **Внутренний** — нельзя создать без экземпляра внешнего. Держит ссылку на внешний объект → риск утечки памяти, если внутренний объект живёт дольше (например, передан в долгоживущий кэш или поток). Правило: если доступ к `Outer.this` не нужен — делай `static`.
- **Локальный и анонимный** — видят только `final` / effectively final локальные переменные. Анонимный класс не может иметь конструктор и реализует ровно один интерфейс или наследует один класс.
- Анонимный класс vs лямбда: в анонимном `this` — сам анонимный объект, в лямбде — внешний; лямбда подходит только для функционального интерфейса.

Перечисления — enum

enum — тип с фиксированным набором именованных экземпляров-констант. Неявно наследует `java.lang.Enum` (поэтому не может наследовать другой класс, но может реализовывать интерфейсы); каждая константа — `public static final` экземпляр.

```
enum Role {
    ADMIN(100), USER(10);                          // константы-объекты

    private final int accessLevel;                 // поле
    Role(int accessLevel) { this.accessLevel = accessLevel; }   // конструктор всегда private
    public int accessLevel() { return accessLevel; }
}

Role r = Role.valueOf("USER");                     // по имени; нет такого — IllegalArgumentException
for (Role x : Role.values()) {                     // массив всех констант
    System.out.println(x.name() + " " + x.ordinal());
}
r == Role.USER;                                    // сравниваем через == — экземпляры единственны
```

Возможности:

- поля, конструкторы, методы, в том числе **абстрактные** — каждая константа реализует их в своём теле:

```
enum Op {
    PLUS  { long apply(long a, long b) { return a + b;
} },
    MINUS { long apply(long a, long b) { return a - b;
} };
    abstract long apply(long a, long b);
}
```

- работает в `switch`; в switch-выражении по enum компилятор проверяет, что перебраны все константы;
- `EnumMap` / `EnumSet` — специализированные коллекции на массиве и битовой маске, быстрее `HashMap` / `HashSet`;
- enum — лучший способ сделать **синглтон**: потокобезопасен, защищён от рефлексии и сериализации.

Не храни `ordinal()` в БД — порядок констант может поменяться; храни `name()`.

Object — equals, hashCode, toString, clone

Object — корневой класс: его наследует любой класс, включая массивы.

Метод	Назначение
`boolean equals(Object o)`	равенство по смыслу; по умолчанию — `this == o`
`int hashCode()`	хэш для хэш-таблиц; по умолчанию — идентификационный хэш, не связанный

Метод	Назначение
	с полями
`String toString()`	строковое представление; по умолчанию `ClassName@hexHash`
`Class<?> getClass()`	класс объекта в рантайме
`protected Object clone()`	копия объекта (см. ниже)
`wait()`, `notify()`, `notifyAll()`	ожидание и сигналы на мониторе объекта — см. **Java Concurrency**
`protected void finalize()`	**deprecated for removal** (Java 9 / 18, JEP 421) — не используем

equals и ==

- `==` — для примитивов сравнивает значения, для ссылок — **адреса** (один ли это объект).
- **`equals()`** — логическое равенство, которое определяет сам класс. `String`, обёртки, коллекции, `record` уже переопределяют его по содержимому.

Контракт `equals` (отношение эквивалентности):

- **рефлексивность** — `x.equals(x) == true`;
- **симметричность** — `x.equals(y) == y.equals(x)`;
- **транзитивность** — `x.equals(y) && y.equals(z)` → `x.equals(z)`;
- **согласованность** — повторные вызовы дают тот же результат, пока объекты не менялись;
- **`x.equals(null) == false`**.

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
// 1. та же ссылка
    if (!(o instanceof Point p)) return false;
// 2. тип + null (null instanceof → false)
    return x == p.x && y == p.y;
// 3. значимые поля
}

@Override
public int hashCode() {
    return Objects.hash(x, y);
// или 31 * x + y
}
```

`getClass()` vs `instanceof`:

- `getClass() != o.getClass()` — равны только объекты **точно** одного класса. Строго соблюдает симметричность при наследовании.
- `instanceof` — наследник может быть равен предку. Симметричность ломается, если наследник добавляет поля в `equals`. Безопасно для `final` классов и когда наследники не меняют `equals`.

Ловушки:

```
public boolean equals(Point that) { ... }   // это
ПЕРЕГРУЗКА, а не переопределение:
                                            // HashSet
и List.contains вызывают equals(Object) — твой метод не
```

```
сработает.
                                                    //
@Override сразу покажет ошибку.

public boolean equals(Object o) { return hashCode() ==
o.hashCode(); }   // неверно: разные объекты могут
иметь одинаковый хэш
```

hashCode и связь с equals

`hashCode()` — целое число, по которому хэш-коллекции (`HashMap`, `HashSet`) выбирают корзину для объекта.

Контракт:

1. если `a.equals(b)` → `a.hashCode() == b.hashCode()` — **обязательно**;
2. одинаковый `hashCode` **не** означает `equals` — коллизии допустимы (значений `int` всего 2^{32}, объектов — бесконечно);
3. пока поля, участвующие в `equals`, не менялись, `hashCode` не меняется.

Отсюда правило: **переопределил `equals` — переопределяй `hashCode`** по тем же полям.

Что сломается, если переопределить только `equals`: два равных объекта получат разные хэши, попадут в разные корзины — `HashSet` хранит дубликаты, `map.get(равныйКлюч)` возвращает `null`.

Обратная ловушка — хэш по части полей корректен, но плох:

```
class Point {
    int x, y;
    public boolean equals(Object o) { ... x == p.x && y
== p.y ... }
    public int hashCode() { return x; }    // контракт
соблюдён, HashSet работает корректно,
}                                          // но все
точки с одним x — в одной корзине → поиск деградирует
```

Почему `31 * x + y` лучше `x + y`: при `x + y` точки `(1,2)` и `(2,1)` дают одинаковый хэш — много коллизий. Умножение на простое нечётное 31 учитывает порядок полей; JIT заменяет его на `(x << 5) - x`.

Какие поля брать: те же, что в `equals`, желательно неизменяемые. Если ключ `HashMap` изменить после вставки — его хэш поменяется и элемент будет «потерян» в старой корзине.

Подробно о том, как `HashMap` использует `hashCode` и `equals`, — в конспекте **Collections**.

toString

Переопределяем для логов и отладки. Не выводим чувствительные данные (пароли, номера карт). `record` генерирует `toString` сам.

Клонирование

Клонирование — создание копии объекта. Механизм `clone()`:

- класс реализует маркерный `Cloneable` (иначе `Object.clone()` бросит `CloneNotSupportedException`);

- переопределяет `clone()` как `public` и вызывает `super.clone()`;
- `Object.clone()` создаёт объект **без вызова конструктора** и копирует поля побитово.

Поверхностное (shallow) — копируются значения полей: примитивы копируются, ссылки — нет, копия указывает на те же вложенные объекты. Это поведение `Object.clone()`.

Глубокое (deep) — рекурсивно копируются и вложенные изменяемые объекты.

```
class Order implements Cloneable {
    List<String> items = new ArrayList<>();

    @Override
    public Order clone() {
        try {
            Order copy = (Order) super.clone();      //
shallow: copy.items == this.items
            copy.items = new ArrayList<>(items);     //
делаем deep для изменяемого поля
            return copy;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);
        }
    }
}
```

`clone()` объявлен в `Object`, а не в `Cloneable`, потому что копирование полей — нативная операция JVM, которой нужен

доступ ко внутренностям любого объекта; `Cloneable` лишь разрешает её.

Какой способ лучше: не `clone()`. Он обходит конструктор, плохо работает с `final`-полями, бросает checked-исключение, по умолчанию поверхностный. Предпочтительно:

- **конструктор копирования** или фабрика `Order.copyOf(other)`;
- **неизменяемые объекты** (`record`, `List.copyOf`) — их вообще не нужно копировать.

Исключение — массивы: `array.clone()` удобен и корректен (поверхностная копия).

finalize — устарел

`finalize()` вызывался GC перед удалением объекта. Проблемы: вызов не гарантирован и непредсказуем по времени, замедляет GC, позволяет «воскресить» объект. Помечен **deprecated for removal**. Вместо него:

- **try-with-resources** + `AutoCloseable` — для освобождения ресурсов;
- `java.lang.ref.Cleaner` (Java 9) — страховка, если ресурс забыли закрыть.

final vs finally vs finalize: `final` — модификатор «нельзя изменить»; `finally` — блок, выполняемый после `try`; `finalize()` — устаревший метод `Object`.

Исключения

Исключение — объект-наследник `Throwable`, сигнализирующий об ошибке; прерывает нормальное выполнение и раскручивает стек вызовов до ближайшего подходящего `catch`.

Иерархия

```
Throwable
├── Error                          — сбой JVM/окружения, не обрабатываем
│   ├── OutOfMemoryError
│   ├── StackOverflowError
│   └── ExceptionInInitializerError, NoClassDefFoundError ...
└── Exception                      — checked (кроме ветки RuntimeException)
    ├── IOException, SQLException, InterruptedException ...
    └── RuntimeException           — unchecked
        ├── NullPointerException
        ├── IllegalArgumentException → NumberFormatException
        ├── IllegalStateException
        ├── IndexOutOfBoundsException
        ├── ClassCastException
        ├── ArithmeticException
        ├── UnsupportedOperationException
        └── ConcurrentModificationException
```

Checked и unchecked

- **Checked** — `Exception` и наследники, кроме `RuntimeException`. Компилятор **требует** обработать (`catch`) или объявить (`throws`). Ожидаемые внешние сбои, от которых можно восстановиться: файл не найден, сеть упала.
- **Unchecked** — `RuntimeException`, `Error` и их наследники. Компилятор не проверяет. Обычно ошибки программиста (NPE, неверный аргумент) или фатальные сбои.

Различие существует только на уровне **компилятора** — JVM их не различает.

Современная практика (Spring и большинство библиотек) — в основном unchecked; checked оборачивают:

```
try {
    Files.readString(path);
} catch (IOException e) {
    throw new UncheckedIOException("Cannot read " +
path, e);   // сохраняем cause!
}
```

`Error` — серьёзные проблемы, которые приложение обычно не может исправить. Ловить их не нужно; исключение — логирование на самом верхнем уровне.

Разновидности `OutOfMemoryError`:

- `Java heap space` — не хватает кучи (утечка или мало `-Xmx`);
- `GC overhead limit exceeded` — GC работает почти всё время и почти ничего не освобождает;
- `Metaspace` — слишком много загруженных классов (часто утечка ClassLoader'ов);

- `unable to create native thread` — ОС не даёт создать поток;
- `Requested array size exceeds VM limit`, `Direct buffer memory`.

throw, throws, своё исключение

- **`throw`** — бросить исключение: `throw new IllegalArgumentException("amount < 0");`
- **`throws`** — в сигнатуре метода: какие checked-исключения он может выбросить, ответственность за обработку переходит вызывающему.

Собственное исключение — наследуем `RuntimeException` (unchecked) или `Exception` (checked), добавляем конструкторы с сообщением и причиной:

```
public class InsufficientFundsException extends
RuntimeException {
    private final long shortage;

    public InsufficientFundsException(long shortage) {
        super("Insufficient funds, shortage: " +
shortage);
        this.shortage = shortage;
    }
    public InsufficientFundsException(String message,
Throwable cause) {
        super(message, cause);
        this.shortage = 0;
    }
```

```
    public long shortage() { return shortage; }
}
```

try / catch / finally

- `try` — код, который может бросить исключение;
- `catch` — обработчик; выполняется **первый** подходящий по типу, остальные пропускаются;
- `finally` — выполняется **всегда**: после `try`, после `catch`, и даже если в них был `return` или новое исключение.

```
try {
    read();
} catch (FileNotFoundException e) {     // сначала частное
    ...
} catch (IOException e) {               // потом общее
    ...
} finally {
    cleanup();
}
```

- Порядок `catch`: **от частного к общему**. `catch (IOException)` перед `catch (FileNotFoundException)` — ошибка компиляции: второй блок недостижим. Выполнится ровно один блок.
- **try-finally без catch** допустим: исключение уйдёт выше, но `finally` выполнится.
- **Multi-catch** (Java 7): `catch (IOException | SQLException e)` — типы не должны быть наследниками друг друга; переменная `e` неявно `final`.

Когда `finally` не выполнится:

- `System.exit()` / `Runtime.halt()` в `try` или `catch`;
- аварийное завершение JVM (crash, `kill -9`, отключение питания);
- бесконечный цикл или дедлок в `try`;
- поток — демон, и JVM завершилась, когда остались только демоны.

Ловушка: `return` в `finally` **перекрывает** `return` из `try` и **глотает** исключение. Не пишите `return` в `finally`.

```
int f() {
    try { throw new RuntimeException(); }
    finally { return 1; }     // исключение потеряно,
вернётся 1
}
```

try-with-resources

try-with-resources (Java 7) — автоматически закрывает ресурсы, реализующие `AutoCloseable`, после выхода из блока.

```
try (var in = Files.newInputStream(src);
     var out = Files.newOutputStream(dst)) {
    in.transferTo(out);
}   // закроются в обратном порядке: out, потом in
```

- Ресурсы закрываются **в обратном порядке** объявления, до `catch` и `finally`.

- Если упал и блок, и `close()` — основным остаётся исключение из блока, а исключение из `close()` добавляется как **suppressed** (`e.getSuppressed()`). В ручном `try-finally` исключение из `close()` затёрло бы основное.
- С Java 9 можно использовать уже объявленную effectively final переменную: `try (conn) { ... }`.

Исключение из main

`main` может объявить `throws Exception`. Непойманное исключение уходит в обработчик потока по умолчанию (`Thread.UncaughtExceptionHandler`): печатается стектрейс в `System.err`, поток `main` завершается. Если других не-демон потоков нет — JVM завершается с кодом 1.

Generics

Generics (обобщения) — параметризация классов, интерфейсов и методов типами. Дают проверку типов при компиляции и избавляют от ручных приведений: `List<String>` вместо `List` + `(String) list.get(0)`.

Обобщённые классы и методы

```
class Pair<K, V> {                                    // параметры типа K, V
    private final K key;
    private final V value;
    Pair(K key, V value) { this.key = key; this.value = value; }
    K key() { return key; }
```

```
    V value() { return value; }
}

Pair<String, Integer> product = new Pair<>("Телефон",
50_000);   // <> — diamond, Java 7
String name = product.key();      // без приведения
int price = product.value();      // автораспаковка

static <T> T firstOrNull(List<T> list) {          //
обобщённый метод: <T> перед возвращаемым типом
    return list.isEmpty() ? null : list.get(0);
}
```

Ограниченный параметр (bounded type) — `T` должен быть подтипом указанного типа; можно несколько ограничений через `&` (класс первым):

```
static <T extends Comparable<T>> T max(List<T> list) {
... }
static <T extends Number & Comparable<T>> T clamp(T v,
T lo, T hi) { ... }
```

Стирание типов (type erasure)

Стирание типов — generics существуют только для компилятора. В байт-коде параметры заменяются на границу (`Object` или `extends`-тип), а компилятор вставляет приведения. В рантайме `List<String>` и `List<Integer>` — один класс `List`.

Следствия — ограничения generics:

```
new T();                          // нельзя — тип T неизвестен в рантайме
new T[10];                        // нельзя создать массив T (и new List<String>[10])
obj instanceof List<String>       // нельзя — только List<?> или List
List<int>                         // нельзя — только ссылочные типы (List<Integer>)
static T field;                   // нельзя — static общий для всех параметризаций
class MyEx<T> extends Exception // нельзя — catch не сможет различить типы
void m(List<String> l) {} void m(List<Integer> l) {}
// нельзя — после стирания одинаковые сигнатуры
```

Raw type — использование generic-класса без параметра (`List list`). Оставлено для совместимости с кодом до Java 5; отключает проверки — не используем.

Инвариантность и wildcards

Generics **инвариантны**: `List<Integer>` **не** является `List<Number>` , хотя `Integer` — подтип `Number` . Иначе можно было бы положить `Double` в список `Integer` .

Массивы, наоборот, **ковариантны** — и поэтому небезопасны:

```
Object[] arr = new String[1];
arr[0] = 1;                        // компилируется, в рантайме ArrayStoreException
```

Wildcard `?` — неизвестный тип; гибкость там, где нужна вариантность:

Запись	Что можно передать	Читать	Писать
`List<?>`	список чего угодно	как `Object`	только `null`
`List<? extends Number>`	`List<Integer>`, `List<Double>` …	как `Number`	нельзя
`List<? super Integer>`	`List<Integer>`, `List<Number>`, `List<Object>`	как `Object`	`Integer` и его подтипы

PECS — Producer Extends, Consumer Super:

- коллекция **отдаёт** элементы (producer, мы читаем) → `? extends T`;
- коллекция **принимает** элементы (consumer, мы пишем) → `? super T`.

```
// из JDK: копируем из src (producer) в dest (consumer)
public static <T> void copy(List<? super T> dest,
List<? extends T> src)

double sum(Collection<? extends Number> nums) {
// читаем — extends
    double s = 0;
    for (Number n : nums) s += n.doubleValue();
    return s;
}
void fill(List<? super Integer> out) {
// пишем — super
```

```
    out.add(42);
}
```

Java 8 — лямбды, функциональные интерфейсы, Optional, Date-Time API

Java 8 (2014, LTS) — самый крупный релиз языка: в Java пришло функциональное программирование.

Главное, что добавили:

- **лямбда-выражения** и **ссылки на методы**;
- **функциональные интерфейсы** (`java.util.function`);
- **`default`- и `static`-методы интерфейсов** — см. Интерфейс;
- **Stream API** — см. конспект **Java Stream**;
- **`Optional`** ;
- **Date-Time API** (`java.time`);
- новые методы `Map` и коллекций (`forEach` , `removeIf` , `computeIfAbsent` , `merge` ...);
- `CompletableFuture` , `ConcurrentHashMap` на CAS, `StampedLock` , `LongAdder` — см. **Java Concurrency**;
- **PermGen удалён**, метаданные классов — в **Metaspace** (нативная память) — см. **JVM**;
- `@Repeatable` -аннотации и аннотации на типах, `String.join` / `StringJoiner` , `Base64` , беззнаковая арифметика (`Integer.toUnsignedString`), `Arrays.parallelSort` .

Устарело: JS-движок **Nashorn** и утилита `jjs` удалены в Java 15.

Лямбда-выражения

Лямбда-выражение — анонимная функция: компактная реализация единственного абстрактного метода функционального интерфейса.

```
(параметры) -> выражение
(параметры) -> { операторы; return ...; }

Comparator<String> byLen = (a, b) ->
Integer.compare(a.length(), b.length());
Runnable r = () -> System.out.println("run");
Function<Integer, Integer> sq = x -> x * x;          //
скобки у одного параметра не нужны
BiFunction<Long, Long, Long> sum = (var a, var b) -> a
+ b;   // var в параметрах — Java 11

list.sort((a, b) -> a.compareTo(b));                 //
сортировка списка строк
list.sort(Comparator.naturalOrder());                //
то же короче
```

Доступ к переменным:

- поля класса и статические поля — читать и менять можно;
- локальные переменные внешнего метода — только `final` / effectively final (лямбда захватывает **копию** значения);
- `this` внутри лямбды — внешний объект (в отличие от анонимного класса).

Под капотом лямбда не создаёт отдельный `.class`: компилятор генерирует приватный метод, а объект интерфейса создаётся в

рантайме через `invokedynamic` + `LambdaMetafactory`.

Ссылки на методы

Ссылка на метод — сокращение лямбды, которая просто вызывает существующий метод.

Вид	Синтаксис	Эквивалент лямбды
Статический метод	`Integer::parseInt`	`s -> Integer.parseInt(s)`
Метод конкретного объекта	`System.out::println`	`x -> System.out.println(x)`
Метод произвольного объекта типа	`String::toUpperCase`	`s -> s.toUpperCase()`
Конструктор	`ArrayList::new`	`() -> new ArrayList<>()`

`System.out::println` — ссылка на метод `println` конкретного объекта `System.out` (статическое поле типа `PrintStream`): получает элемент и печатает его.

Функциональные интерфейсы

Функциональный интерфейс — интерфейс ровно с **одним абстрактным методом** (`default`, `static` и методы `Object` не считаются). Его экземпляр можно задать лямбдой или ссылкой на метод. Аннотация `@FunctionalInterface` заставляет компилятор это проверить.

Базовые интерфейсы `java.util.function`:

Интерфейс	Метод	Смысл	Пример
`Function<T,R>`	`R apply(T)`	преобразовать	`String::l`
`UnaryOperator<T>`	`T apply(T)`	`Function<T,T>`	`String::t`
`BinaryOperator<T>`	`T apply(T,T)`	свернуть два в один	`Integer::`
`Predicate<T>`	`boolean test(T)`	проверить условие	`String::i`
`Consumer<T>`	`void accept(T)`	потребить, побочный эффект	`System.ou`
`Supplier<T>`	`T get()`	поставить значение	`ArrayList`
`BiFunction<T,U,R>`, `BiPredicate<T,U>`, `BiConsumer<T,U>`	—	то же для двух аргументов	`Map::put`

Специализации для примитивов (без автоупаковки):

- `IntFunction<R>`, `IntPredicate`, `IntConsumer`, `IntSupplier`, `IntUnaryOperator`, `IntBinaryOperator` — и такие же с `Long`, `Double`; `BooleanSupplier`;
- `ToIntFunction<T>`, `ToLongBiFunction<T,U>` ... — возвращают примитив;
- `IntToLongFunction`, `DoubleToIntFunction` ... — примитив → примитив;
- `ObjIntConsumer<T>` ... — объект + примитив.

Композиция:

```
Predicate<String> notBlank =
Predicate.not(String::isBlank);   // Java 11
Predicate<String> valid = notBlank.and(s -> s.length()
< 64);
Function<String, String> strip = String::strip;
Function<String, String> norm =
strip.andThen(String::toLowerCase);
Comparator<User> cmp =
Comparator.comparing(User::lastName).thenComparing(User
::age).reversed();
```

Optional

`Optional<T>` — контейнер, который содержит значение или пуст. Явно говорит в сигнатуре: «результата может не быть» — вместо возврата `null`.

```
Optional<User> user = repo.findById(id);

String name = user.map(User::name).orElse("unknown");
User u = user.orElseThrow(() -> new
NotFoundException(id));   // orElseThrow() без
аргументов — Java 10
user.ifPresentOrElse(this::send, this::logMissing);
// Java 9
```

- `orElse(x)` вычисляет `x` **всегда**, `orElseGet(() -> x)` — только если пусто.
- Используем как **возвращаемый тип**. Не используем для полей, параметров методов и в коллекциях.

- `Optional.get()` без проверки — по сути тот же NPE (`NoSuchElementException`); лучше `orElseThrow()`.

Подробнее — в конспекте **Java Stream**, раздел «Optional как результат стрима».

Новые методы Map

```
Map<String, Integer> counts = new HashMap<>();

counts.getOrDefault("a", 0);
// значение или дефолт
counts.putIfAbsent("a", 1);
// положить, если ключа нет
counts.merge(word, 1, Integer::sum);
// нет ключа → 1, есть → старое + 1 (подсчёт частот)
counts.computeIfAbsent(key, k -> new ArrayList<>
()).add(v);   // мультимапа
counts.computeIfPresent("a", (k, v) -> v + 1);
// вернёт null → ключ удалится
counts.compute("a", (k, v) -> v == null ? 1 : v + 1);
counts.forEach((k, v) -> System.out.println(k + "=" +
v));
counts.replaceAll((k, v) -> v * 2);
counts.entrySet().removeIf(e -> e.getValue() == 0);  //
removeIf у коллекций
```

Фабрики неизменяемых коллекций — уже **Java 9**: `List.of`, `Set.of`, `Map.of`.

Date-Time API (java.time)

Date-Time API — замена старым `java.util.Date` и `Calendar` (изменяемые, не потокобезопасные, месяцы с 0). Все классы `java.time` **неизменяемые** и потокобезопасные.

Класс	Что хранит	Когда
`LocalDate`	дата без времени и зоны: `2026-10-14`	дни рождения, даты операций
`LocalTime`	время без даты	расписание
`LocalDateTime`	дата + время без зоны	локальные события «на стене»
`ZonedDateTime`	дата + время + часовой пояс (`Europe/Moscow`)	события для пользователя в его зоне
`OffsetDateTime`	дата + время + смещение (`+03:00`)	API, БД (`timestamptz`)
`Instant`	точка на временной шкале UTC (наносекунды от эпохи)	хранение и логирование времени
`Duration` / `Period`	промежуток в секундах / в днях-месяцах-годах	таймауты / сроки

```
LocalDate today = LocalDate.now();
today.plusWeeks(1).plusMonths(1).plusYears(10);
// каждая операция — новый объект

today.with(TemporalAdjusters.next(DayOfWeek.TUESDAY));
// следующий вторник
today.with(TemporalAdjusters.dayOfWeekInMonth(2,
DayOfWeek.SATURDAY));   // вторая суббота месяца
```

```java
LocalTime.now().truncatedTo(ChronoUnit.MILLIS);
// время с точностью до мс
ZonedDateTime.now(ZoneId.of("Europe/Moscow"));
Instant.now();

Duration.between(start, end).toMillis();
Period.between(birth, today).getYears();

DateTimeFormatter f =
DateTimeFormatter.ofPattern("dd.MM.yyyy");   //
потокобезопасный, в отличие от SimpleDateFormat
LocalDate.parse("14.10.2026", f);
```

Практика бэкенда: храним и передаём **`Instant`** / **`OffsetDateTime`** **в UTC**, в зону пользователя переводим только при отображении. Для тестов внедряем `Clock` вместо прямого `now()`.

Прочее из Java 8

```java
String.join(", ", List.of("a", "b"));
// "a, b"
new StringJoiner(", ", "[", "]").add("a").add("b");
// "[a, b]"

String enc = Base64.getEncoder().encodeToString(bytes);
// также getUrlEncoder(), getMimeEncoder()
byte[] dec = Base64.getDecoder().decode(enc);

@Repeatable(Roles.class)
// повторяемая аннотация
@interface Role { String value(); }
```

```
@interface Roles { Role[] value(); }
// контейнер
@Role("ADMIN") @Role("USER") class Service { }
```

Java 9–25 — var, record, sealed, pattern matching, text blocks

С Java 9 релизы выходят каждые полгода, LTS — раз в два года: **11, 17, 21, 25**. Фичи обычно сначала идут как preview, потом финализируются — ниже версии **финального** релиза.

Версия	Что важное появилось в языке и API
9	модули (JPMS), `List.of` / `Set.of` / `Map.of`, `private`-методы интерфейсов, `jshell`
10	`var` для локальных переменных
11 (LTS)	`String.isBlank/strip/lines/repeat`, `HttpClient`, `java Main.java`, `var` в параметрах лямбды
14	switch-выражения `->`, понятные сообщения NPE (helpful NPE)
15	text blocks `"""`
16	`record`, pattern matching для `instanceof`, `Stream.toList()`
17 (LTS)	`sealed`-классы
21 (LTS)	pattern matching в `switch`, record patterns, **virtual threads**, `SequencedCollection` (`getFirst()`, `reversed()`)
22	безымянные переменные `_`

Версия	Что важное появилось в языке и API
24	Stream Gatherers
25 (LTS)	компактные исходники и `void main()`, код до `super(...)` в конструкторе, `import module`

var

`var` (Java 10) — вывод типа локальной переменной из правой части. Тип определяется при компиляции и не меняется — это не динамическая типизация.

```
var accounts = new HashMap<String, List<Account>>();
// тип очевиден справа — хорошо
for (var e : accounts.entrySet()) { }
try (var in = Files.newInputStream(path)) { }

var x;                      // нельзя: нет инициализатора
var n = null;               // нельзя: тип не вывести
var list = new ArrayList<>();   // выведется
ArrayList<Object> — ловушка
```

Только для локальных переменных, не для полей, параметров и возвращаемых типов. Используем, когда тип очевиден из правой части.

record

record (Java 16) — класс-носитель неизменяемых данных. Компилятор генерирует: `private final` поля, канонический конструктор, методы доступа `name()` (без `get`), `equals`, `hashCode`, `toString`.

```
public record Money(BigDecimal amount, String currency)
{
    public Money {
// компактный конструктор — валидация
        Objects.requireNonNull(currency);
        if (amount.signum() < 0) throw new
IllegalArgumentException("negative");
    }
    public Money add(Money o) { return new
Money(amount.add(o.amount), currency); }
    public static Money zero(String c) { return new
Money(BigDecimal.ZERO, c); }
}

var m = new Money(new BigDecimal("10.00"), "EUR");
m.amount();
// accessor
```

Ограничения:

- неявно `final` и наследует `java.lang.Record` — нельзя наследовать ни его, ни им;
- нельзя добавить нестатические поля сверх компонентов;
- можно: реализовывать интерфейсы, статические поля и методы, свои методы, вложенные типы.

Неизменяемость **поверхностная**: `record R(List<String> l)` не защищает сам список — делай `l = List.copyOf(l)` в компактном конструкторе.

Применение: DTO, value objects, ключи `Map`, результаты методов из нескольких значений.

sealed

sealed-класс / интерфейс (Java 17) — явно перечисляет, кто может его наследовать. Иерархия становится закрытой — компилятор знает все варианты.

```
public sealed interface Payment permits CardPayment,
BankTransfer, Refund { }

public record CardPayment(String pan, long amount)
implements Payment { }
public record BankTransfer(String iban, long amount)
implements Payment { }
public non-sealed class Refund implements Payment { }
// ветка снова открыта для наследования
```

Каждый наследник обязан быть `final` (record — неявно), `sealed` или `non-sealed`. Наследники — в том же модуле (или пакете, если модулей нет).

Главная польза — **исчерпывающий** `switch` без `default`: добавили новый тип в `permits` — компилятор покажет все места, где его забыли обработать.

switch-выражения и pattern matching

switch-выражение (Java 14) — `switch` возвращает значение; стрелки `->` не проваливаются в следующий case (нет fall-through), `break` не нужен.

Pattern matching — проверка типа и извлечение данных одной конструкцией:

- `instanceof` с переменной (Java 16);
- паттерны типов в `switch` и условия `when` (Java 21);
- **record patterns** — деконструкция record прямо в паттерне (Java 21).

```
if (obj instanceof String s && !s.isBlank()) {   // s доступна дальше в условии и в теле
    use(s);
}

long fee(Payment p) {
    return switch (p) {
// исчерпывающий для sealed — без default
        case CardPayment c when c.amount() > 10_000 -> 0;     // guard
        case CardPayment c   -> c.amount() / 100;
        case BankTransfer(String iban, long amount) -> 50;    // record pattern
        case Refund r        -> 0;
    };
}

String describe(Object o) {
    return switch (o) {
        case null        -> "null";            // null можно обработать явно (иначе NPE)
        case Integer i   -> "int " + i;
        case String s    -> "str " + s;
        default          -> "other";
    };
}
```

Безымянные переменные (Java 22) — когда значение не нужно: `case BankTransfer(_, long amount) -> ...`, `catch (Exception _)`.

Text blocks

Text block (Java 15) — многострочный строковый литерал в `"""`. Общий отступ убирается автоматически, кавычки не нужно экранировать.

```
String json = """
        {
          "id": %d,
          "status": "%s"
        }
        """.formatted(id, status);

String sql = """
        SELECT * FROM accounts \
        WHERE id = ?""";          // \ в конце строки — без переноса
```

Модули (JPMS)

Модуль (Java 9) — именованная группа пакетов с файлом `module-info.java`: явно объявляет зависимости (`requires`) и экспортируемые пакеты (`exports`). Даёт сильную инкапсуляцию — неэкспортированные пакеты недоступны даже через рефлексию без `opens`.

```
module com.bank.payments {
    requires java.sql;
```

```
    exports com.bank.payments.api;          // доступно
другим модулям
    opens com.bank.payments.model;          // открыто
для рефлексии (Jackson, Hibernate)
}
```

На практике в Spring-приложениях модули почти не используют (classpath), но JDK сам разбит на модули — отсюда ошибки `InaccessibleObjectException` при рефлексии во внутренности JDK.

Reflection и аннотации

Reflection

Reflection (рефлексия) — API (`java.lang.reflect`), позволяющий в рантайме исследовать и менять структуру классов: получать поля, методы, конструкторы, аннотации, вызывать методы и создавать объекты по имени, обходить `private` .

```
Class<?> clazz = Class.forName("com.bank.Account");
// загрузка класса по имени
Class<?> c2 = Account.class;
// или так
Class<?> c3 = account.getClass();
// или от объекта

Object obj =
clazz.getDeclaredConstructor().newInstance();
```

```java
Field f = clazz.getDeclaredField("balance");
// getDeclared* — все члены этого класса
f.setAccessible(true);
// доступ к private
f.set(obj, 100L);

Method m = clazz.getMethod("deposit", long.class);
// get* — только public, включая унаследованные
m.invoke(obj, 50L);
```

На рефлексии построены фреймворки: Spring (DI, `@Autowired`), Hibernate (маппинг полей), Jackson (JSON), JUnit (поиск `@Test`), а также динамические прокси (`java.lang.reflect.Proxy`).

Минусы: медленнее прямого вызова, нет проверок компилятора, ломает инкапсуляцию; модули (Java 9+) запрещают доступ к закрытым пакетам без `opens`.

Динамическая загрузка классов — `Class.forName(...)` загружает класс в рантайме через ClassLoader (так раньше подключали JDBC-драйверы; сейчас это делает `ServiceLoader`). Устройство ClassLoader'ов и модель делегирования — в конспекте **JVM**.

Аннотации

Аннотация — метаданные, прикреплённые к элементу кода (класс, метод, поле, параметр, тип). Сама ничего не делает — её читает компилятор, процессор аннотаций при сборке или код через рефлексию в рантайме.

Встроенные: `@Override`, `@Deprecated(forRemoval = true)`, `@SuppressWarnings`, `@FunctionalInterface`, `@SafeVarargs`.

Своя аннотация:

```
@Retention(RetentionPolicy.RUNTIME)          //
доступна через рефлексию
@Target(ElementType.METHOD)                  // куда
можно ставить
public @interface Audited {
    String action();                         //
элемент; типы: примитивы, String, Class, enum,
аннотации, массивы
    boolean logArgs() default false;
}

@Audited(action = "TRANSFER")
public void transfer(...) { }

// обработка
for (Method m : service.getClass().getMethods()) {
    Audited a = m.getAnnotation(Audited.class);
    if (a != null) log(a.action());
}
```

`@Retention` — сколько живёт аннотация:

- `SOURCE` — только в исходнике, отбрасывается компилятором (`@Override`, Lombok);
- `CLASS` — в `.class`, но недоступна в рантайме (по умолчанию);
- `RUNTIME` — доступна через рефлексию (`@Autowired`, `@Transactional`, `@Entity`).

`@Target` — `TYPE`, `METHOD`, `FIELD`, `PARAMETER`, `CONSTRUCTOR`, `TYPE_USE` (Java 8)... Также есть `@Inherited` (наследуется подклассами) и `@Repeatable`.

Прочее — varargs, assert

Varargs

Varargs — параметр переменной длины `Type... name`; внутри метода это обычный массив.

```
static long sum(long... values) {            // values — long[]
    long s = 0;
    for (long v : values) s += v;
    return s;
}
sum();  sum(1);  sum(1, 2, 3);  sum(new long[]{1, 2});

String.format("%s=%d", "a", 1);              // Object... args
```

- Только **один** varargs-параметр и только **последним**.
- При перегрузке varargs выбирается в последнюю очередь — после точного совпадения, расширения и автоупаковки.
- Generic varargs (`T... items`) создают массив стёртого типа → предупреждение «heap pollution»; если метод безопасен — `@SafeVarargs`.
- Каждый вызов создаёт массив — в горячем коде JDK держит перегрузки на 0–N аргументов (как `List.of`).

assert

`assert` — проверка инварианта при разработке: `assert условие : "сообщение";`. Если условие ложно — `AssertionError`.

```
assert balance >= 0 : "balance must not be negative: "
+ balance;
```

По умолчанию **выключены** — включаются флагом `-ea` (`java -ea App`). Поэтому `assert` нельзя использовать для проверки аргументов и бизнес-правил — для этого `Objects.requireNonNull`, `IllegalArgumentException`. В продакшен-коде встречается редко, в тестах используют assertions JUnit / AssertJ.

Числа и точность — BigDecimal

Почему double нельзя для денег

`float` и `double` — двоичные числа с плавающей точкой (IEEE 754). Большинство десятичных дробей (`0.1`, `0.2`) в двоичном виде бесконечны и хранятся с округлением.

```
0.1 + 0.2;              // 0.30000000000000004
0.1 + 0.2 == 0.3;       // false
1.10 - 1.00;            // 0.10000000000000009

Double.NaN == Double.NaN;         // false — NaN не
равен ничему; проверка: Double.isNaN(x)
```

```
0.0 == -0.0;                        // true, но
Double.compare(0.0, -0.0) == 1
Math.abs(a - b) < 1e-9;             // сравнение double —
с допуском
```

Для денег два варианта: `BigDecimal` или `long` **в минимальных единицах** (копейки, центы) — быстрее, но нужно помнить про масштаб валюты.

BigDecimal

BigDecimal — неизменяемое десятичное число произвольной точности: `unscaledValue × 10^(-scale)`. `12.30` = `1230` со scale `2`.

```
new BigDecimal("0.1");              // точно 0.1 —
создаём из строки
BigDecimal.valueOf(0.1);            // тоже 0.1 — через
Double.toString
new BigDecimal(0.1);                //
0.1000000000000000055511151231257827... — НЕ так

BigDecimal total = price.multiply(qty).add(fee);   //
каждая операция — новый объект
total = total.setScale(2, RoundingMode.HALF_EVEN); //
масштаб и округление
```

equals vs compareTo:

```
new BigDecimal("2.0").equals(new BigDecimal("2.00"));
// false — учитывает scale
```

```
new BigDecimal("2.0").compareTo(new
BigDecimal("2.00"));   // 0 — сравнивает значение
```

Сравниваем через `compareTo`. В `HashSet` / ключах `HashMap` `2.0` и `2.00` — разные ключи; нормализуй `stripTrailingZeros()` или фиксированным `setScale`.

Деление:

```
BigDecimal.ONE.divide(BigDecimal.valueOf(3));
// ArithmeticException: бесконечная дробь
BigDecimal.ONE.divide(BigDecimal.valueOf(3), 2,
RoundingMode.HALF_UP);   // 0.33
```

RoundingMode:

- `HALF_UP` — школьное округление: `2.5 → 3`;
- `HALF_EVEN` — банковское: к ближайшему чётному, `2.5 → 2`, `3.5 → 4` — не накапливает смещение на больших объёмах;
- `DOWN` / `UP` — к нулю / от нуля, `FLOOR` / `CEILING` — вниз / вверх по числовой оси.

Для больших целых — `BigInteger`.

Вопросы к собеседованию

Платформа и типы — вопросы

Чем отличаются JDK, JRE и JVM?

JVM исполняет байт-код; JRE = JVM + стандартная библиотека (запуск); JDK = JRE + инструменты разработки (`javac` , `jar` ...). С

Java 11 отдельный JRE не поставляется.

Какие примитивные типы есть в Java?

`byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean` — 8 штук. Всё остальное — ссылочные типы.

Правда ли, что примитивы всегда в стеке, а объекты в куче?

Нет. Объекты — в куче, но примитивные поля объекта лежат в куче вместе с ним. В стеке — только локальные переменные и параметры методов (ссылки и примитивы).

В Java передача по значению или по ссылке?

Всегда по значению. Для объектов копируется ссылка: объект изменить можно, подменить его у вызывающего — нет.

Что выведет `Integer a = 127, b = 127; a == b` и то же для 128?

`true` и `false`: `Integer.valueOf` кэширует −128…127. Обёртки сравниваем через `equals`.

Чем `&&` отличается от `&`?

`&&` — сокращённое: правая часть не вычисляется, если левая `false`. `&` на `boolean` вычисляет обе части, на целых — побитовое И.

Что такое автоупаковка и где она может бросить NPE?

Автоматическое `int ↔ Integer` через `valueOf` / `intValue`. NPE — при распаковке `null` (в т.ч. в тернарнике и арифметике с обёртками).

String — вопросы

Почему String неизменяемый?

Пул строк, безопасность (строки как параметры подключения и имена классов), потокобезопасность, кэширование `hashCode` → идеальный ключ `HashMap`.

Что такое пул строк и intern()?

Область в куче с единственными экземплярами литералов. `intern()` возвращает экземпляр из пула, добавляя строку, если её там нет.

Сколько объектов создаёт `new String("abc")`?

До двух: литерал `"abc"` в пуле (если его там ещё не было) и новый объект в куче.

String vs StringBuilder vs StringBuffer?

`String` неизменяемый; `StringBuilder` изменяемый, не синхронизирован — для сборки строк в цикле; `StringBuffer` синхронизирован, на практике устарел.

Почему пароль лучше хранить в char[]?

Массив можно затереть сразу после использования; строка остаётся в памяти до сборки мусора.

Классы и ООП — вопросы

Какие модификаторы доступа есть?

`private` (класс), package-private (пакет), `protected` (пакет + наследники), `public` (все).

Что означает final для класса, метода, поля, переменной?

Класс нельзя наследовать, метод — переопределить, поле и переменную — переприсвоить. Объект по `final`-ссылке менять можно.

Можно ли переопределить статический метод?

Нет — только скрыть (hiding); вызов выбирается по типу ссылки при компиляции. Перегрузить — можно.

Можно ли сделать метод одновременно abstract и static?

Нет: абстрактный должен быть переопределён, а статический переопределить нельзя.

Каков порядок инициализации при создании объекта наследника?

static-блоки и поля родителя → static наследника (один раз) → поля и блоки родителя → конструктор родителя → поля и блоки наследника → конструктор наследника.

Что будет при исключении в static-блоке?

`ExceptionInInitializerError`, а при следующих обращениях к классу — `NoClassDefFoundError`.

Абстрактный класс или интерфейс — что выбрать?

Интерфейс — по умолчанию, для роли и множественной реализации. Абстрактный класс — когда нужны общее состояние, конструктор, `protected`-методы.

Что можно менять при переопределении метода?

Доступ — только расширять; возвращаемый тип — тот же или подтип; checked-исключения — только убирать или сужать. Параметры менять нельзя (это будет перегрузка).

Чем статический вложенный класс отличается от внутреннего?

Внутренний хранит ссылку на экземпляр внешнего и без него не создаётся; статический вложенный — нет. Если ссылка на внешний не нужна — делай `static` (меньше памяти, нет риска утечки).

Какие переменные видят лямбды и анонимные классы?

Поля — любые; локальные — только `final` / effectively final.

Как сделать неизменяемый класс?

`final` класс, `private final` поля, без сеттеров, защитное копирование изменяемых полей. Или `record` + `List.copyOf`.

Object — вопросы

Какой контракт у equals и hashCode?

`equals`: рефлексивность, симметричность, транзитивность, согласованность, `equals(null) == false`. Равные по `equals` объекты обязаны иметь равные `hashCode`; обратное не требуется.

Что будет, если переопределить equals без hashCode?

Равные объекты попадут в разные корзины: дубликаты в `HashSet`, `HashMap.get` не находит ключ.

Могут ли разные объекты иметь одинаковый hashCode?

Да — это коллизия; `int` ограничен 2^{32} значениями.

getClass() или instanceof в equals?

`getClass()` — строгая симметричность при наследовании; `instanceof` — допускает равенство с наследниками, безопасен для `final`-классов и с pattern matching короче.

Почему `equals(MyClass o)` — ошибка?

Это перегрузка, а не переопределение: коллекции вызывают `equals(Object)`. `@Override` ловит такую ошибку.

Поверхностное vs глубокое клонирование, и что лучше clone()?

Поверхностное копирует ссылки на вложенные объекты, глубокое — сами объекты. Вместо `clone()` — конструктор

копирования или неизменяемые объекты.

Чем отличаются final, finally и finalize?

Модификатор «нельзя изменить»; блок, выполняемый после `try`; устаревший метод `Object`, который вызывал GC.

Исключения — вопросы

Checked vs unchecked?

Checked (`Exception` кроме `RuntimeException`) — компилятор требует обработать или объявить. Unchecked (`RuntimeException`, `Error`) — нет.

Всегда ли выполняется finally?

Почти: кроме `System.exit()`, падения JVM, бесконечного цикла / дедлока в `try`, завершения JVM при живых только потоках-демонах.

Что будет, если в finally есть return?

Он перекроет результат `try` и проглотит исключение.

В каком порядке писать catch для IOException и FileNotFoundException?

Сначала `FileNotFoundException` (подкласс), потом `IOException`; иначе ошибка компиляции. Выполнится один блок.

Что даёт try-with-resources?

Автоматический `close()` в обратном порядке; исключение из `close()` не затирает основное, а добавляется в suppressed.

Generics — вопросы

Что такое стирание типов и к каким ограничениям оно ведёт?

Параметры типов есть только при компиляции; в байт-коде — `Object` / граница. Поэтому нельзя `new T()`, массив `T[]`, `instanceof List<String>`, примитивы в параметрах, перегрузка по разным параметризациям.

Почему `List<Integer>` нельзя передать как `List<Number>`?

Generics инвариантны: иначе в список `Integer` можно было бы положить `Double`. Для гибкости — `List<? extends Number>`.

Что такое PECS?

Producer Extends, Consumer Super: читаем из коллекции — `? extends T`, пишем — `? super T`.

Java 8+ — вопросы

Что такое функциональный интерфейс?

Интерфейс с одним абстрактным методом; реализуется лямбдой или ссылкой на метод. `Function`, `Predicate`, `Consumer`, `Supplier`, `UnaryOperator`, `BinaryOperator`.

Какие виды ссылок на методы бывают?

На статический метод, на метод конкретного объекта, на метод произвольного объекта типа, на конструктор.

orElse vs orElseGet?

`orElse` вычисляет аргумент всегда, `orElseGet` — только если `Optional` пуст.

LocalDateTime vs ZonedDateTime vs Instant?

Дата-время без зоны; с часовым поясом; точка на шкале UTC. Хранить время — `Instant` / UTC.

Что такое record и чем он ограничен?

Неизменяемый носитель данных с генерацией конструктора, accessors, `equals/hashCode/toString`. Нельзя наследовать и наследоваться, нельзя добавлять поля экземпляра; неизменяемость поверхностная.

Зачем нужны sealed-классы?

Закрытая иерархия: компилятор знает всех наследников → исчерпывающий `switch` без `default` и проверка при добавлении нового типа.

Числа — вопросы

Почему нельзя считать деньги в double?

Десятичные дроби не представимы точно в двоичном виде: `0.1 + 0.2 != 0.3`. Используем `BigDecimal` или `long` в копейках.

Почему `new BigDecimal("2.0").equals(new BigDecimal("2.00"))` — false?

`equals` учитывает scale. Значения сравниваем через `compareTo`.

Почему не `new BigDecimal(0.1)`?

Конструктор принимает уже неточный `double` → `0.1000000000000000055...`. Создаём из строки или `BigDecimal.valueOf`.